HLTCOE Generation Team at TREC 2025

Kevin Duh,[†] Dawn Lawrie,[†] Debashish Chakraborty,[†] Roxana Petcu,[*]
Eugene, Yang,[†] Kenton Murray,[†] Daniel Khashabi,[‡] Maxime Dassen[*]
[†]Human Language Technology Center of Excellence, Johns Hopkins University, USA
[‡]Johns Hopkins University, USA, [*]University of Amsterdam, Netherlands
kevinduh@cs.jhu.edu,{lawrie,dchakra6,kenton}@jhu.edu,danielk@cs.jhu.edu,r.m.petcu@uva.nl,m.s.w.dassen@uva.nl

ABSTRACT

The HLTCOE generation team experimented with different frameworks for various Retrieval-Augmented Generation (RAG) tasks in the BioGen, RAG, and RAGTIME tracks. Based on RAGTIME dry run results, an extractive approach that focuses on the facts expressed in documents is effective at citation accuracy, while being competitive with other approaches at covering the required facts, leading to the highest performing system in RAGTIME dryruns. Among the systems that approach the task from a summarization perspective, more agentic approaches outperform cascade systems that examine fewer documents. When faced with multilingual documents, we find that using machine-translated documents as the source for generating responses improves the citation ability of the system.

KEYWORDS

GPT-Researcher, LangGraph, AutoGen, document context, RAG

1 INTRODUCTION

This year the primary contribution of the HLTCOE's Generation Team was in a diverse set of frameworks used for generation. Key focuses across all generation approaches included how to best represent the content of the document in a prompt to enable faithful generation and faithful citation of source material.

Regardless of the approach, all generation utilized the same large language model (LLM), `llama-3.3-70b-instruct`, unless otherwise noted. The approaches also have similar prompts for the different aspects of the process, although there are slight variations across the approaches. In addition, the same retrieval engine was used for most of the submitted runs.

Many of the design decisions were influenced by the multilingual RAGTIME task, with development being done on the NeuCLIR 2024 Pilot Report Generation Task [10]. For RAGTIME, all the retrieval was done in a multilingual fashion, where with queries in English and documents indexed in their native languages. The retrieval engine had to successfully rank documents in diverse languages. On the generation side, English translations of the documents were generally used. We found that citation accuracy was negatively impacted if documents appeared in multiple languages in the prompt [8].

In addition to the RAGTIME generation task, runs were submitted to the RAG and BioGen Tracks. Once results become available, results of the submissions across the three tracks will be compared to see if top performing systems are consistent.

2 REPORT GENERATION SYSTEMS

We developed systems based on four main frameworks: GPT-Researcher, LangGraph, AutoGen, and Extractive Generation using DSPy. We consider GPT-Researcher and the extractive approach to be cascade systems since all retrieval is done before generation. LangGraph and AutoGen are more agentic frameworks and retrieve additional information to augment the initial information retrieved. We also experimented some variations that were integrated into particular frameworks. In GPT-Researcher, we followed Shao et al. [12] to create an interview process for generating search queries. In LangGraph, we experimented with Conformal Prediction as a means of improving the salience of the generated output, using only 8B Models in the generation process, and explicitly checking citations with the ARGUE prompt [14]. The same citation checking process was used as a stand-alone process for AutoGen. Finally, we investigated an alternative citation improving approach within the extractive approach which we called *Citogenesis*.

All of our approaches interacted with a search service in order to issue queries that would retrieve documents. While it is becoming clear that RAG is extremely dependent on the quality of retrieval, our focus was the generation process, so we did not explore any re-ranking approaches. That exploration can be found in Yates et al. [17]. Therefore, we used first-stage retrievers, generally fusing a single-vector dense retriever, a multi-vector dense retriever, and a learned-sparse retrieval approach. Our single-vector dense retriever was `Qwen3-8B` [18]. Our multi-vector dense retriever was `PLAID-X` [15]. For English, our learned-sparse retrieval approach was `Splade-V3` [9], while our multilingual retrieval approach was `MILCO` [11]. While requiring more storage since it relied on three different indexes of the documents, it was quite fast at runtime since no re-ranker was used. Reciprocal rank fusion was used to combine multiple retrieved rankings. For runs that did use a fusion of all three approaches, indicated by "Fusion" in the run description appearing in Table 1. Otherwise, the retriever is named explicitly. Occasionally we combined only two retrievers using reciprocal rank fusion. In those cases, we list the two names with a + between them.

In general, improving citation accuracy was one of the main focuses across all of our approaches. We found that using the document collection id was more likely to lead to hallucinated ids. Therefore, during generation we used the numbers 1-N. Only when parsing the generation to create the final output required by the track did we replace these temporary ids with the actual document collection ids.

2.1 GPT-Researcher Approach

This summarization-based approach used a language model with the initial topic as input, generated a persona for that topic, split the topic into a variable number of subqueries to find information, then aggregated that information using a language model. The prompt was tuned to enforce an unbiased tone and strictly limit citations to sources that explicitly support the generated claims, thereby reducing the rate of irrelevant or hallucinated citations. GPT-Researcher[1] was used as the framework for this approach; we improved upon our system at LiveRAG 2025 [3] in the following ways.

First, we manually optimized the generation prompt for `llama` through iterative testing. As seen in Figure 1, to ensure adherence to the formatting constraints (such as forcing every sentence onto a new line), we employed 'emotional pleading' techniques. An example of this is the instruction 'this is very important to my career', which has been observed to improve instruction-following. We also explicitly constrained citation placement to the end of sentences to prevent the model from breaking the flow of text or halluncinating citations mid-sentence. The output is post-processed to extract citations and sentences in the structured format required by the task.

Second, we investigated different ways in which documents are represented in the prompt, i.e. {processed_document} variable in Figure 1: (1) raw document in its entirety, (2) filtered snippets using embedding similarity with the subquery [3], and (3) summarization using a "notetaking" LLM as shown in Figure 2. We found that the performance generally improves with compressed representations, even if the generation LLM can support long contexts.

In order to stay within the length limit of the task, we included the length in the instructions. If the track's length limit was expressed in characters, we estimated the number of words by dividing the length limit by five, our estimate of the average length of a word. If the generation is too long, the end of the response is truncated.

All runs that used this approach are listed as the approach `GPT-R` in Table 1. In addition to the retrieval and compression approaches, the number of queries the system is expected to generate as well as the number of documents whose content is considered is fixed. These hyperparameters are identified with each run. The variant we considered with this approach is described next.

2.1.1 Interviewing to Create Queries. The initial query decomposition stage in the basic GPT-Researcher approach formulates K subqueries based on the problem statement, background, and documents from an initial search. In contrast, this approach, referred to as *interview*, utilizes two LLMs: one serves as an interviewer and one as an interviewee. The LLMs complete a few rounds of conversation to generate the subqueries. In each round, the interviewer is asked to generate a set of subqueries designed to probe details about the topic. The interviewee responds by generating answers to the questions asked by the interviewer, pushing for detailed information needs. The process runs for multiple rounds, effectively generating a large set of both exploratory subqueries that seek new information, and clarifying subqueries for previously surfaced information needs. This is inspired by the STORM system [12]. Run 13 in Table 1 completes one round of interviews. Run 14 completes three rounds of interviews.

Information:
Source 1: {processed_document}
Source 2: {processed_document} ...
—
Using the above information, provide supporting facts for the query: "{problem statement}" in a detailed report – The report should focus on the answer to the query, should be well structured, informative, and contain ALL facts and numbers in the supporting information in at most {N} words.

Please follow all of the following guidelines in your report:
- You MUST determine your own concrete and valid opinion based on the given information. Do NOT defer to general and meaningless conclusions.
- You MUST write the report in the following language: English with each sentence on its own line.
- You MUST cite your sources, especially for relevant sentences that answer the question.
- When using information that comes from the documents, use inline notation which refer to the Document ID (e.g. [1] for Source: 1).
- It is important to ensure that the Document ID is a valid string from the information above and that the information in the sentence is present in the document.
- Do not include a list of citations at the end of the document.
- Do not put citations in the middle of the sentence. Always cite at the end of the sentence.
- Write the report in a Objective (impartial and unbiased presentation of facts and findings) tone.
Please do your best, this is very important to my career.

Figure 1: GPT-Researcher generation prompt for RAGTIME

2.2 LangGraph Researcher Approach

This approach uses the LangGraph framework[2] to decompose the generation problem into a variety of steps that work together to implement a RAG system, as shown in Figure 3. The main difference between this approach and GPT-Researcher to the ability of the system to repeat steps when desired, making it a more agentic approach. The workflow starts with a query generation step that generates a configurable number of queries. Each of those subqueries is issued to a retrieval step, which works in parallel to retrieve documents. The retrieval step returns a configured number of documents from a search service specified in the configuration. Each of the retrieved documents is then compressed in some way. This determines the context that is used during generation. Two main strategies for context compression were experimented with in TREC 2025 runs. Generally, an approach referred to as note taking

[1] https://github.com/assafelovic/gpt-researcher

[2] https://github.com/langchain-ai/langgraph

Table 1: Runs submitted to the different tracks

Run id	Run Name	Approach	Retrieval	Compression	# loops	# queries per loop	# docs per query	Use MT docs	other
RAG, Full RAG Task									
1	gptr_nt_q3d3	GPT-R	Splade-V3	notetaking	1	3	3	-	
2	gptr.nt_q4d4	GPT-R	Splade-V3	notetaking	1	4	4	-	
3	gptr_e2_q3d3	GPT-R	Splade-V3	sim	1	3	3	-	
4	gptr_e2_q4d4	GPT-R	Splade-V3	sim	1	4	4	-	
5	lg_nt_q4d12l3_c	LangGraph	Splade-V3	notetaking	3	4	12	-	*fix cite*
6	lg_nt_q4d12l3	LangGraph	Splade-V3	notetaking	3	4	12	-	
7	swarm	AutoGen	Splade-V3	notetaking	-	3	6	-	
8	swarm_c	AutoGen	Splade-V3	notetaking	-	3	6	-	*fix cite*
9	extractive_rag	Extractive	Splade-V3	notetaking	1	10	3	-	
RAGTIME, Dry Run									
10	embed.mt_q3d3	GPT-R	Fusion	sim	1	3	3	yes	
11	keep_all.mt_q3d3	GPT-R	Fusion	keep all	1	3	3	yes	
12	keep_all.src_q3d3	GPT-R	Fusion	keep all	1	3	3	no	
13	high-to-low-interview-1-rrf	GPT-R	Fusion	keep all	1	6	10	yes	*interview*-1
14	high-to-low-interview-3-rrf	GPT-R	Fusion	keep all	1	6	10	yes	*interview*-3
15	lg-w-ret_plq-nt-s_cite-4q.3l.5r	LangGraph	Fusion	notetaking	3	4	5	yes	
16	lg-w-ret_lq-nt-s_cite-4q.3l.5r	LangGraph	MILCO+Qwen3-8B	notetaking	3	4	5	yes	
17	lsr_qwen.concise_prompts. 2loop.7docs	LangGraph	MILCO+Qwen3-8B	*pred*	2	4	7	yes	
18	bulleted_list	Extractive	Fusion	notetaking	1	10	3	yes	
19	bulleted_list_plus_rewrite	Extractive	Fusion	notetaking	1	10	3	yes	*Citogenesis*
RAGTIME									
20	gptr_nt_q3d3_mt	GPT-R	Fusion	notetaking	1	3	3	yes	
21	gptr_e2_q3d3_mt	GPT-R	Fusion	sim	1	3	3	yes	
22	gptr_ka_q3d3_mt	GPT-R	Fusion	keep all	1	3	3	yes	
23	gptr_ka_q3d3_natv	GPT-R	Fusion	keep all	1	3	3	no	
24	gptr_nt_q4d4_mt	GPT-R	Fusion	notetaking	1	4	4	yes	
25	lg_nt_4q12r3l_mt_c	LangGraph	Fusion	notetaking	3	4	12	yes	*fix cite*
26	lg_nt_4q12r3l_natv_c	LangGraph	Fusion	notetaking	3	4	12	no	*fix cite*
27	lg_e2_3q5r3l	LangGraph	Fusion	sim	3	3	5	yes	
28	lg_e2_3q5r2l_mt_qw3	LangGraph	Fusion	sim	2	3	5	yes	*8B Models*
29	auto_swarm_mt	AutoGen	Fusion	notetaking	-	3	6	yes	
BioGen									
30	llama70B.lg-w-ret_lpq-nt-s_cite	LangGraph	Fusion	notetaking	2	4	12	-	
31	llama70B.lg-w-ret_p-nt-s_cite	LangGraph	PLAID-X	notetaking	2	4	12	-	

is used. This approach prompts an LLM to extract facts from the document that are responsive to the user's request for information. Another approach chunks the document into overlapping snippets and uses embedding similarity between the snippet and sub-query to determine if the snippet should be part of the context. Each snippet whose score is above some threshold is added to the context. Herein 0.6 is used as the threshold. No matter which approach is used, all retained text is concatenated with document identifiers attached to form the context for generating the output. Before generating a response, the reflection node reviews all the context to determine if any information is missing. If there are gaps, it generates more queries for search. This reflection step is one of the main differences between this approach and GPT-Researcher. When no more information is required (or the maximum allowed loops is reached), a response is drafted based on the context. Unlike GPT-R, no length limit is imposed on the generation.

Since all tracks impose a limit on the generation, the revision step ensures the generation does not exceed the length limit. Our approach first uses an LLM to shorten each individual sentence. We felt this would be the most robust approach since it does not run the risk of combining information from multiple documents,

Extract at most ten citable important facts that should be included in a report that responds the user request.
Extract ONLY facts that are worth mentioning to the user based on the request.
Our user has NO patience for reading irrelevant information and does NOT care about missing information.
ONLY what you found is important.
DO NOT include non-factoid information.
Also, DO NOT repeat similar facts.
Put a short description of each fact in one line without any formatting.
Use at most 20 words to describe each facts.
If the document contains no relevant facts related to the user request, output the string "N/A".

Request: {question}
Document: {raw_document}

Figure 2: Prompt for Notetaking LLM step

a property we like to avoid in order to ensure that each sentence is fully supported by the document(s) it cites. If the generation continues to be too long, then the entire generation is revised with a prompt to the LLM. Three attempts are made to satisfy the length limit before giving up and truncating the end of the response.

In Table 1 a run that uses this approach to RAG has `LangGraph` in the "Approach" column. With this approach, we created three variants. One of these variants focused on improving citations during the revision step indicated with *fix cite* in the "Other" column. A second variant investigated the performance when limiting the RAG process to smaller models indicated with *8B Models* in the "Other" column. The third variant investigated an alternative compression approach indicated with *pred* in the "Compression" column.

2.2.1 Improving Citations. The citation fix was inserted into the Revise Response Step as shown in Figure 3. After sentences are shortened but before whole-sale revision is attempted, each citation is checked using the ARGUE citation checking prompt [14]. If it is determined that the cited document does not attest the information in the sentence, other retrieved documents are examined to see if any of them attest the sentence. If another document is found, the citation is updated to the new document. Otherwise the sentence is removed. The reason that it was inserted between the two revision approaches is that citation checking can lead to a shorter generation that no longer exceeds the length limit.

2.2.2 Generating with Small Models. The goal of Run 28 in Table 1 was to investigate whether smaller models could be effective in a RAG task. We targeted the RAGTIME task for this experiment. While the Llama-72B continued to be used for query generation and reflection, two 8B models were used for compression and for writing the generated response. While all similarity comparisons were performed with `qwen-8B-embedding`, we fine-tuned `qwen-8B-instruct` using LoRA (Low-Rank Adaptation) [5] to generate the response.

To produce training data, we used `llama3.3 70b-instruct` to generate a set of 500 unique dummy report requests. A report request consisting of a background and problem statement was combined with a set of retrieved documents and a generated report. For each report request, we created a training example with both the machine-translated (MT) and untranslated documents, producing a total of 1000 training examples. For hyperparameters, we set $\alpha = 16$ and $r = 64$, which fell in the recommended range from the QLoRA paper [2], and dropout to 0.1. All linear layers were fine-tuned, allowing for full-finetuning level performance [2]. For training parameters, the following were used:

Param	Batch Size	Gradient Steps	Optimizer	Epochs	Learn Rate	Warmup Steps
Value	1	2	AdamW	1	2e-4	10

2.2.3 Conformal Prediction. We integrated conformal prediction [1, 13] to learn statistically-grounded filtering thresholds for document relevance. While the filtering method uses cosine similarity between query and document embeddings, the threshold value of 0.5566 was derived through conformal prediction calibration rather than manual tuning.

The calibration process began by constructing a dataset of query-document snippet pairs from the NeuCLIR 2024 report generation dataset. For each pair, we used an LLM-based entailment checker to verify whether the snippet supported the query's information need, creating a labeled calibration set. We computed nonconformity scores as $1 - \text{cosine\_similarity}(q, d)$ for each query-snippet pair, where lower scores indicate higher relevance. The conformal threshold was then computed as the $(1 - \alpha)$ percentile of these calibration scores, providing statistical coverage guarantees. For Run 17 in Table 1, we used $\alpha = 0.1$ (90% coverage), which yielded a conformal threshold that corresponds to a cosine similarity threshold of 0.5566.

In the LangGraph pipeline, conformal filtering occurs in the Search and Compress Documents Step in Figure 3. Documents are first split into overlapping snippets (chunk size 1000 characters, overlap 100 characters), and each snippet's cosine similarity to the query is computed using Qwen3-Embedding-8B embeddings. Snippets with similarity scores above the learned threshold of 0.5566 are retained, while those below are filtered out. This snippet-level filtering approach allows for finer-grained relevance decisions compared to document-level filtering, as it can retain relevant portions of partially relevant documents while removing irrelevant sections.

The system generated 3 initial queries in the first research loop. After retrieval and filtering, the reflection node analyzed the gathered information and generated additional follow-up queries (typically 2-4 queries) for the second loop, addressing identified knowledge gaps. The maximum number of research loops was set to 2, ensuring the system completes within a reasonable time while still allowing for iterative refinement.

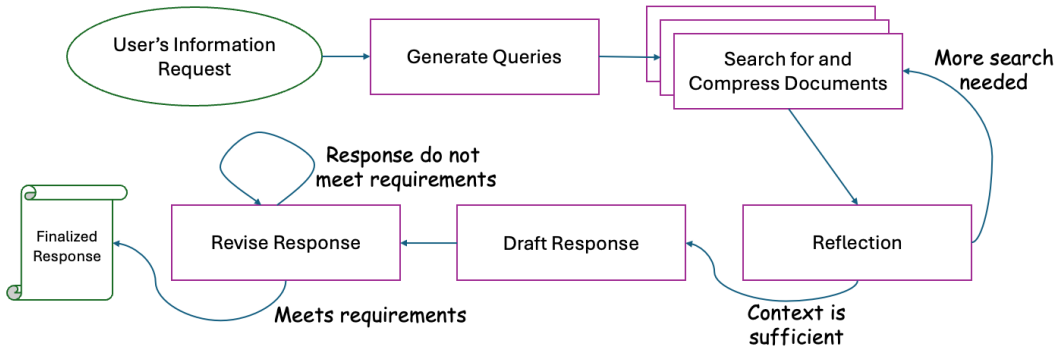

Figure 3: Steps followed by the LangGraph Approach to produce a response

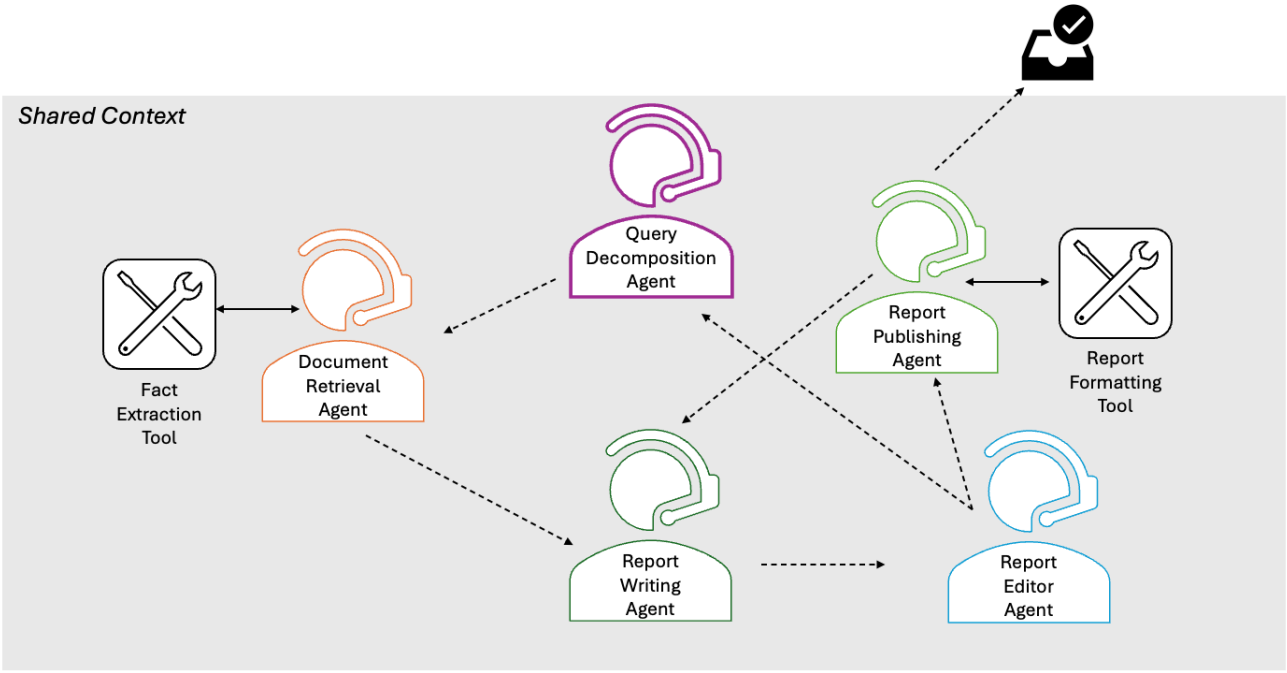

Figure 4: Agents available to AutoGen to write a response

2.3 AutoGen Researcher Approach

The AutoGen Framework[3] implements an agentic approach to RAG. Five agents as outlined in Figure 4 take turns to craft a response to a user's information request. By default an LLM is used to select the next agent to take a turn based on the list of available agents and the history of what has happened so far. An agent can request that another agent be the next agent to take a turn. The dotted lines in Figure 4 reflect which agents could be suggested as potential agents to follow. Responding with the next agent to follow is part of the LLM generation by the agent and may fail. Some agents have tools available to them. The retrieval agent uses a fact extraction tool, while the report publishing agent uses the report formatting tool.

Generally, the process begins by selecting the query decomposition agent, which decomposing the user request into up to three queries. The queries are sent to the document retriever agent, which initiates the search. For the six top ranked documents, a list of facts are extracted that address the system generated query, not the original user's information request. The Report Writing Agent assembles the fact from the documents, removing duplicates. Then the Report Editor Agent is invoked. The purpose of the editor is to identify knowledge gaps in the information assembled so far. If

[3]https://microsoft.github.io/autogen/stable/index.html

a knowledge gap is identified, the gap is described and the Query Decomposition Agent is invoked. If there is no knowledge gap, the Report Publishing Agent is called. The Report Publishing Agent provides constructive feedback. If the feedback is satisfied, the Report Formatting Tool is called which formats the output into the appropriate JSON object and replaces temporary document ids with their legitimate ids. Then the agent responds to "APPROVE", which triggers the end of the task. However, if the publishing agent is unsatisfied, it returns the report to the Report Writing Agent so that further edits may be made.

Since this approach ignored any length limit, the revisions setup in `LangGraph` is used a standalone process to shorten the generation. In addition, the fixing citations as described in Section 2.2.1 can also be involved.

Runs that use the AutoGen approach to RAG are identified by `AutoGen` in Table 1. The "# loops" is not given because the LLM must decide to end the process.

2.4 Extractive Approach

The extractive approach is very similar to the extractive approach used in the NeuCLIR Report Generation Pilot in 2024 [16]. Despite conceptually being extractive, the pipeline uses the language model to "extract" information instead of using a traditional BIO-style extraction model. In this approach, we use the framework DSPy[6, 7].

Similar to the `GPT-R`, the extractive approach also first splits the topic into multiple queries. Specifically, the generative model is prompted to generate ten "Google-like" queries based on the report request. The queries are then issued to a search service to obtain the top three documents for each query.

For each retrieved document, we prompted the generative model to extract key facts based on the report requests, similar to notetaking. The model then groups the extracted facts into twenty "canonical" facts from all retrieved documents with the document ID being attached to each group.

Given that the RAGTIME evaluation metrics prioritize individual sentence support and nugget recall over narrative coherence, we optimized our system to generate canonical facts rather than a flowing narrative.

This approach is identified as `Extractive` in Table 1. Given that this approach groups facts, it is possible to lose track of which document attests the information appearing in the sentence. To address this issue, we created a post-processing step to improve citation called *Citogenesis*, which is described next.

2.4.1 Citogenesis. Run 19 in Table 1 is motivated by the fact that report generation via Retrieval Augmented Generation (RAG) requires thorough and reliable citation of all sources to combat the risk of hallucination. Many report generation frameworks rely on their LLM to track source documents by ID; however, this in itself is also prone to hallucination, especially when dealing with long contexts [4] where the LLM can "lose track" of which document IDs correspond to which statements.

To reduce the uncertainty and risk inherent to generating a cited report all at once, we assign citations to a finished report text by comparing generated sentences individually to sets of facts extracted from source documents in order to reduce the occurrence of sentences without applicable citations.

Citation assignment relies on LLM notetaking to generate lists of individual claims or facts, with each list corresponding to a unique retrieved document. The uncited body of a report is then generated based on those fact lists. Finally, each sentence in this generated report is compared with each recorded fact to determine which facts contain information that attests or supports each sentence. Since each fact instance is mapped to exactly one document, facts that attest a given sentence correspond to appropriate documents. This system still uses LLM queries to determine citations, but alters the nature of the task from positional token or word tracking within a large context window to semantic evaluation of a small amount of input with essentially boolean output, a task for which LLMs are generally more explicitly suited.

3 RESULTS

Runs were submitted to three tracks. Currently we have only received results for the dry run task within the RAGTIME Track. Those results are discussed below

3.1 RAGTIME Track

We submitted runs to two tasks within the RAGTIME Track, the dry run task and the main task.

Table 2: Results of RAGTIME, Dry Run. Bold values are the match the maximum value, while italicized values are about the median, which is always higher than the mean.

Run id	Sentences	Sentence Support	Nugget Coverage	F1
max	431	0.930	0.468	0.602
median	267	0.824	0.394	0.513
10	186	0.710	0.294	0.398
11	178	*0.893*	*0.399*	*0.535*
12	183	0.782	*0.395*	0.507
13	194	0.682	0.347	0.453
14	195	0.772	0.351	0.462
15	307	*0.885*	*0.441*	*0.575*
16	289	*0.848*	*0.467*	*0.579*
17	322	*0.891*	*0.405*	*0.544*
18	368	*0.920*	**0.468**	**0.602**
19	167	0.648	0.380	0.445

3.1.1 Dry Run Task. Table 2 reflects systems that were run between July 2 and July 8 of 2025. Therefore, despite some of the names being the same, these are not the same systems submitted later in the summer. Among Runs 10, 11, and 12 using `GPT-R`, keeping the entire document instead of using embeddings to select snippets leads to higher Sentence Support and Nugget Coverage. While using the native document or the machine-translated document has little effect on Nugget Coverage, Sentence Support is much higher when using machine-translated documents. Between Runs 13 and 14, three rounds of interviews is slightly better than one;

however, not utilizing the interviews leads to better performance as represented by Run 11. Using LangGraph in Runs 15 and 16 leads to improvements over GPT-R. The only difference in those runs comes from the search service. This indicates that Sentence Support, which should be fairly consistent, has high variability. The fusion of MILCO+Qwen3-8B is leads to slightly higher Nugget Coverage. While Conformal Prediction in Run 17 appears to be helpful for improving Sentence Support, it's Nugget Coverage suffers. Finally, the extractive summary continues to be a very strong baseline with the top value for Sentence Support and essentially performing the same as LangGraph in terms of Nugget Coverage. Unfortunately, *Citogenesis* was not an effective approach to improving citations as represented by Sentence Support, especially given it was starting with such a strong system.

Table 3: Results of RAGTIME, Multilingual Report Generation. Bold values match the maximum value, while italicized values are about the median.

Run id	Sentences	Sentence Support	Nugget Coverage	F1
max	429	0.973	0.445	0.529
median	175	0.715	0.340	0.442
20	163	*0.865*	*0.342*	*0.459*
21	142	*0.853*	0.325	0.440
22	140	*0.835*	0.329	0.438
23	142	*0.801*	*0.341*	*0.457*
24	167	*0.820*	*0.354*	*0.466*
25	*240*	*0.885*	*0.398*	*0.524*
26	*258*	*0.894*	*0.407*	**0.529**
27	*246*	*0.850*	*0.382*	*0.495*
28	160	0.251	0.230	0.174
29	*219*	*0.944*	*0.360*	*0.479*

3.1.2 Main Task. Table 3 contains results submitted to the main Multilingual Report Generation task. From the preliminary results, there continues to be support for the notion that citation accuracy as measured by sentence support is harmed by information expressed in multiple languages during report writing as represented by Run 23. While Run 26 also utilized native documents, the model was prompted to produce notes in English. This was a successful approach leading to the highest scoring system in terms of F1. That said using the machine translated documents, while holding everything else constant performed almost the same in terms of averages.

For GPT-R, the nugget coverage and F1 scores tend to be around the median, while sentence support is well above the median. When looking at the different approached to dealing to document context, notetaking (Runs 20 and 24) and increasing the number of documents retrieved and examined (Run 24) gives an advantage in terms of nugget coverage and F1, but provides a more mixed results in terms of sentence support.

For LangGraph, notetaking provides a clear advantage in all three measures as represented by Run 25 and 26 compared to Runs 27 and 28. Although these runs also attempt to fix bad citations using the Auto-ARGUE prompt. That is likely what is responsible for the boost to sentence support. The use of an 8B model was ineffective at the task as seen in Run 28.

Run 29 is the single submission of AutoGen Researcher. It is more accurate than the other two approaches at Sentence Support, even without using the Auto-ARGUE prompt to fix citations; however, it's nugget coverage is not quite as high.

Table 4: BioGen Results. Bold values match the maximum value, while italicized values are about the median.

Run ID	Answer Accuracy	Answer Precision	Answer Recall
max	30	0.7155	0.3767
median	30	0.5761	0.2467
30	**30**	0.5452	*0.3467*
31	**30**	*0.6379*	**0.3767**

3.2 RAG Track

For the RAG, we submitted runs to the Full RAG Task. These results are not yet available.

3.3 BioGen Track

The two submissions to BioGen both used LangGraph. The differences in the runs were in the differnce first stage retrievers. While Answer Accuracy did not differentiate systems, Answer Quality as represented by Precision and Answer Recall as represented by Recall (with Supported and Required)-ST were both higher for the PLAID-X retriever than the Fusion Retriever. Perhaps the more technical nature of the document set is more challenging for LSR and Qwen3 to represents than PLAID-X, which matches query tokens to document tokens in it MaxSim approach to document scoring. The PLAID-X retriever using the open source model llama-70B achieved the top Answer Recall score.

4 CONCLUSION

Given the results of RAGTIME, the Extractive Approach continues to be very effective with highest nugget coverage for the dry-runs and the main multilingual report generation task. Comparing the Extractive Approach to LangGraph Research reveals the power to fixing citations, a step added to the main task. While the Extractive Approach has higher sentence support than LangGraph in the dry runs, LangGraph has much superior sentence support in the main task[4] leading to LangGraph having the highest F1 value over all submitted runs. AutoGen excels at sentence support, but lags behind LangGraph in terms of nugget coverage. GPT-R is quite strong at Sentence Support, but lags behind other approaches in terms of nugget coverage in both RAGTIME dry runs and main task, perhaps because it incorporates fewer documents.

[4]The coordinators submitted the Extractive Approach to the main task under the run name extractive-rag.

REFERENCES

[1] Anastasios Nikolas Angelopoulos and Stephen Bates. 2021. A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. *ArXiv* abs/2107.07511 (2021). https://api.semanticscholar.org/CorpusID:235899036

[2] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLORA: efficient finetuning of quantized LLMs. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 441, 28 pages.

[3] Kevin Duh, Eugene Yang, Orion Weller, Andrew Yates, and Dawn Lawrie. 2025. HLTCOE at LiveRAG: GPT-Researcher using ColBERT retrieval. arXiv:2506.22356 [cs.IR] https://arxiv.org/abs/2506.22356

[4] Kelly Hong, Anton Troynikov, and Jeff Huber. 2025. Context Rot: How Increasing Input Tokens Impacts LLM Performance. https://research.trychroma.com/context-rot

[5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. *CoRR* abs/2106.09685 (2021). arXiv:2106.09685 https://arxiv.org/abs/2106.09685

[6] Omar Khattab, Keshav Santhanam, Xiang Lisa Li, David Hall, Percy Liang, Christopher Potts, and Matei Zaharia. 2022. Demonstrate-Search-Predict: Composing Retrieval and Language Models for Knowledge-Intensive NLP. *arXiv preprint arXiv:2212.14024* (2022).

[7] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. In *The Twelfth International Conference on Learning Representations*.

[8] Dayeon Ki, Marine Carpuat, Paul McNamee, Daniel Khashabi, Eugene Yang, Dawn Lawrie, and Kevin Duh. 2025. Linguistic Nepotism: Trading-off Quality for Language Preference in Multilingual RAG. arXiv:2509.13930 [cs.CL] https://arxiv.org/abs/2509.13930

[9] Carlos Lassance, Hervé Déjean, Thibault Formal, and Stéphane Clinchant. 2024. SPLADE-v3: New baselines for SPLADE. *arXiv preprint arXiv:2403.06789* (2024).

[10] Dawn Lawrie, Sean MacAvaney, James Mayfield, Paul McNamee, Douglas W Oard, Luca Soldaini, and Eugene Yang. 2025. Overview of the TREC 2024 NeuCLIR Track. *arXiv preprint arXiv:2509.14355* (2025).

[11] Thong Nguyen, Yibin Lei, Jia-Huei Ju, Eugene Yang, and Andrew Yates. 2025. MILCO: Learned Sparse Retrieval Across Languages via a Multilingual Connector. *arXiv preprint 2510.00671* (2025).

[12] Yijia Shao, Yucheng Jiang, Theodore Kanell, Peter Xu, Omar Khattab, and Monica Lam. 2024. Assisting in Writing Wikipedia-like Articles From Scratch with Large Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Kevin Duh, Helena Gomez, and Steven Bethard (Eds.). Association for Computational Linguistics, Mexico City, Mexico, 6252–6278. https://doi.org/10.18653/v1/2024.naacl-long.347

[13] Vladimir Vovk, Alex Gammerman, and Glenn Shafer. 2005. *Algorithmic Learning in a Random World*. Springer-Verlag, Berlin, Heidelberg.

[14] William Walden, Marc Mason, Orion Weller, Laura Dietz, John Conroy, Neil Molino, Hannah Recknor, Bryan Li, Gabrielle Kaili-May Liu, Yu Hou, Dawn Lawrie, James Mayfield, and Eugene Yang. 2025. Auto-ARGUE: LLM-Based Report Generation Evaluation. arXiv:2509.26184 [cs.IR] https://arxiv.org/abs/2509.26184

[15] Eugene Yang, Dawn Lawrie, and James Mayfield. 2024. Distillation for Multilingual Information Retrieval. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2368–2373.

[16] Eugene Yang, Dawn Lawrie, Orion Weller, and James Mayfield. 2025. HLTCOE at TREC 2024 NeuCLIR Track. arXiv:2510.00143 [cs.IR] https://arxiv.org/abs/2510.00143

[17] Andrew Yates, Eugene Yang, and Trevor Adriaanse. 2023. HLTCOE Retrieval Team at TREC 2025. In *The Thirty-second Text REtrieval Conference (TREC 2023) Proceedings*.

[18] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *arXiv preprint arXiv:2506.05176* (2025).